Query & Database Optimization - Concept Dictionary

A reference glossary of every key term used in the optimization exercises. Entries are grouped by topic area and listed alphabetically within each section.

Execution Plans

EXPLAIN A PostgreSQL command that displays the execution plan the query planner chose for a statement, without actually running it. Shows estimated costs, row counts, and the sequence of operations (nodes) the engine will perform. Useful for understanding how PostgreSQL intends to execute a query before committing to it.

EXPLAIN ANALYZE Extends `EXPLAIN` by actually executing the query and recording real timing and row counts alongside the planner's estimates. The output shows both `(cost=... rows=... width=...)` (estimates) and `(actual time=... rows=... loops=...)` (measured). Use this to detect cases where the planner's estimates are far from reality.

EXPLAIN (ANALYZE, BUFFERS) Adds buffer (I/O page) statistics to the EXPLAIN ANALYZE output. Reports how many shared memory pages were hit from cache (`shared hit`) vs. read from disk (`shared read`). Essential for measuring the I/O savings from clustering or covering indexes.

Execution Plan (Query Plan) The step-by-step strategy the database engine will follow to execute a SQL statement. Represented as a tree of plan nodes, each performing one operation (scan, join, sort, aggregate, etc.). The planner generates multiple candidate plans and selects the one with the lowest estimated cost.

Plan Node A single operation within an execution plan. Common nodes include:

- `Seq Scan` - reads every row of a table
- `Index Scan` - uses an index to locate rows, then fetches them from the heap
- `Index Only Scan` - satisfies the query entirely from the index, without heap access
- `Bitmap Heap Scan` - collects a bitmap of matching pages from an index, then fetches those pages in order
- `Hash Join` - builds a hash table from one input and probes it with a second
- `Merge Join` - joins two pre-sorted inputs by stepping through them in parallel
- `Nested Loop` - for each outer row, scans the inner side
- `Sort` - sorts rows by specified columns
- `Aggregate` / `HashAggregate` - computes aggregate functions (SUM, COUNT, AVG, etc.)

cost=start..total The two-part cost estimate shown in every plan node. `start` is the estimated cost before the first output row can be produced (relevant for nodes like Sort that must consume all input first). `total` is the estimated cost to produce all output rows. Units are arbitrary planner cost units, not milliseconds. Lower total cost = plan the planner prefers.

rows= The planner's estimated number of rows a node will produce. Compared against `actual rows=` in EXPLAIN ANALYZE output. Large discrepancies indicate stale statistics; fix with `ANALYZE`.

loops= In EXPLAIN ANALYZE output, the number of times a plan node was executed. The `actual time` and `actual rows` figures are per-loop averages. Total rows = `rows * loops`. For the inner side of a Nested Loop, `loops` equals the number of outer rows.

width= The estimated average row size in bytes. Used by the planner to estimate memory requirements (e.g., for hash tables and sort buffers).

Planner (Query Optimizer) The component of PostgreSQL responsible for transforming a parsed SQL statement into an efficient execution plan. It generates candidate plans, estimates their cost using table statistics, and selects the cheapest one. It uses column histograms, table sizes, and configuration parameters (`random_page_cost`, `work_mem`, etc.) to make its estimates.

Statistics (Planner Statistics) Column-level metadata collected by `ANALYZE` and stored in the `pg_statistic` catalog. Includes: number of distinct values, most common values with their frequencies, and a histogram of value distribution. The planner uses these to estimate how many rows a predicate will match. Stale statistics cause poor plan choices.

ANALYZE *(command)* Collects fresh statistics for a table or specific columns and updates `pg_statistic`. Should be run after large data loads or major updates. PostgreSQL's autovacuum daemon runs `ANALYZE` automatically, but manual runs are needed after bulk inserts in exercises.

Selectivity A measure of how many rows a predicate is expected to return, expressed as a fraction (0 to 1) of the total rows. A predicate with high selectivity (e.g., `WHERE order_id = 42`) matches very few rows - this favors index usage. A predicate with low selectivity (e.g., `WHERE status = 'active'` where 95% of rows are active) matches most rows - the planner often prefers a sequential scan.

Scan Types

Sequential Scan (Seq Scan) Reads every page of a table from start to finish, evaluating the WHERE predicate for each row. Preferred when a large fraction of rows match the predicate, the table is very small, or no suitable index exists. The cost is proportional to the number of pages in the table.

Index Scan Uses a B-tree (or other) index to find the TIDs (physical row locations) matching the predicate, then fetches each corresponding row from the heap. Preferred when the predicate is highly selective (few matching rows). Each row requires at least one heap page fetch in addition to the index traversal.

Index Only Scan Like an Index Scan, but all columns needed by the query are stored in the index itself (either as key columns or via `INCLUDE`). PostgreSQL retrieves values directly from the index without touching the heap. Very efficient for read-heavy workloads. Requires that the heap's visibility map is up to date (maintained by VACUUM).

Bitmap Heap Scan Two-phase scan. First, a `Bitmap Index Scan` traverses the index and builds an in-memory bitmap of which heap pages contain matching rows. Then `Bitmap Heap Scan` fetches those pages in physical order (reducing random I/O). Useful when many rows match a predicate - more efficient than many separate heap fetches from an Index Scan, but less so than the Index Only Scan for very selective queries.

Join Strategies

Hash Join One relation (the **build side**, usually the smaller one) is scanned and loaded into an in-memory hash table keyed on the join column. The other relation (the **probe side**) is then scanned row by row; each row is hashed and looked up in the hash table. Efficient for large unsorted relations with no usable index on the join key.

Merge Join Both inputs are sorted (or pre-sorted via an index) on the join column. A single synchronized scan merges them. Very efficient when both sides are already sorted or when large sorted index scans are available. Bad if sorting itself is expensive.

Nested Loop Join For every row in the outer relation, the inner relation is scanned (or index-scanned) for matching rows. Efficient when the outer side is small and the inner side has an index on the join column. Degrades to O(N × M) without an index.

Build Side / Probe Side Terms used in Hash Join. The **build side** is read first and loaded into a hash table in memory. The **probe side** is then streamed through the hash table. The planner typically chooses the smaller relation as the build side to minimize memory usage.

Query Rewriting

Correlated Subquery A subquery that references a column from the outer query. It executes once for every row produced by the outer query. This can be extremely slow for large outer result sets. Often rewritable as a JOIN or window function with dramatically better performance.

CTE (Common Table Expression) A named subquery defined with the `WITH` keyword. In PostgreSQL 12+, CTEs that are non-recursive and do not call volatile functions are **inlined** by default - the planner treats them as if they were subqueries and can optimize them together with the main query. CTEs can be forced to materialize with the `MATERIALIZED` keyword.

CTE Inlining The default behavior in PostgreSQL 12+ where a plain CTE is expanded into the main query before planning. The planner can then push filters and optimize across CTE boundaries. Produces the same plan as embedding the CTE's SQL directly in the outer query.

Materialized CTE A CTE declared with `AS MATERIALIZED (...)`. PostgreSQL executes it first, stores the result in a temporary buffer, and then runs the outer query against that buffer. The CTE is an **optimization fence**: the planner cannot push conditions from the outer query into it. Sometimes faster if the CTE result is reused multiple times; usually slower otherwise.

Predicate Pushdown An optimization where the planner moves a filter condition earlier in the plan - closer to the data source - so fewer rows flow through subsequent operations. An example is pushing `WHERE is_premium = TRUE` inside a CTE when it is inlined, rather than filtering after the CTE materializes all rows.

EXISTS A predicate that returns `TRUE` as soon as any matching row is found in the subquery. It **short-circuits** - it stops scanning as soon as the first match is found. Faster than `COUNT(*) > 0` for checking existence because `COUNT` must process all matching rows.

Short-circuit evaluation Stopping a computation as soon as the result is certain. `EXISTS` short-circuits after the first matching row. `AND` short-circuits if the first condition is `FALSE`. This avoids unnecessary work.

Functional Index (Expression Index) An index built on the result of an expression or function applied to one or more columns, rather than on the raw column values. Example: `CREATE INDEX idx ON users (LOWER(email))`. The query's WHERE clause must use the exact same expression for the index to be used: `WHERE LOWER(email) = 'alice@example.com'`.

Indexes

B-tree Index The default and most general-purpose index type in PostgreSQL. Stores values in a balanced tree structure. Supports equality (`=`), range comparisons (`<`, `>`, `BETWEEN`), `IS NULL`, `IN`, and `ORDER BY`. The standard choice for most indexed columns.

Hash Index An index that supports only equality comparisons (`=`). Implements a hash table mapping each value to its row location. Smaller than B-tree for equality-only workloads. Cannot support range queries or sorting.

GIN Index (Generalized Inverted Index) Best for multi-valued data types where each value contains many elements: arrays, JSONB, full-text search vectors (`tsvector`), and pattern matching (with `pg_trgm`). Inverted means it maps element values back to the rows that contain them.

GiST Index (Generalized Search Tree) A flexible index framework supporting geometric types, range types, full-text search, and nearest-neighbor lookups. Less space-efficient than GIN for text search but supports more query operators (e.g., containment, overlap, distance).

BRIN Index (Block Range Index) Stores the minimum and maximum column value for each range of consecutive heap pages ("block range"). Extremely small and fast to maintain. Effective only when column values are correlated with physical storage order (e.g., a `created_at` timestamp on an append-only table). Much less precise than B-tree - may still require reading many pages per query.

Composite Index (Multi-column Index) An index defined on two or more columns, e.g., `CREATE INDEX ON reviews (game_id, rating)`. The left-prefix rule governs usage:

- A composite index on `(A, B)` can serve queries filtering on `A` alone, or on both `A` and `B`.
- It cannot efficiently serve queries filtering on `B` alone (no left prefix match).
- Column order should reflect the most selective or most commonly filtered column first.

Left-prefix Rule The constraint that a composite index can only be used when the query's WHERE clause includes an equality or range condition on the leftmost column(s) of the index. Queries that skip the leftmost column receive no benefit from the composite index.

Partial Index An index that only includes rows satisfying a specified condition (`WHERE` clause in the index definition). Example: `CREATE INDEX idx ON users (user_id) WHERE account_status = 'suspended'`. Smaller than a full index, cheaper to maintain, and more precise for queries that always include the same filter.

Covering Index An index that contains all columns required to satisfy a query - both for the filter (key columns) and for the output (included columns). Allows Index Only Scan, eliminating heap access entirely. Implemented with the `INCLUDE` clause: `CREATE INDEX idx ON users (country) INCLUDE (total_spent, is_premium)`.

INCLUDE (clause) Added to a `CREATE INDEX` statement to store additional columns in the index leaf pages without making them part of the sort key. These extra columns can be returned by an Index Only Scan without visiting the heap, but they cannot be used as filter or sort keys in the index.

Index Bloat The accumulation of dead index entries that are no longer needed (from deleted or updated rows). Bloated indexes waste space and slow scans. Resolved by `VACUUM`, `REINDEX`, or `REINDEX CONCURRENTLY`.

REINDEX CONCURRENTLY Rebuilds an index without acquiring a lock that blocks reads or writes on the table. Takes longer than a regular `REINDEX` but safe for production systems with live traffic.

pg_stat_user_indexes A PostgreSQL system view that tracks index usage statistics per index, including `idx_scan` (number of times the index was used for a scan). Indexes with `idx_scan = 0` are candidates for removal.

Unused Index An index that has never been used by the query planner (`idx_scan = 0` in `pg_stat_user_indexes`). Unused indexes consume disk space and add overhead to every write operation without providing any read benefit. They should be identified and dropped.

Partitioning

Table Partitioning Dividing a large table into smaller physical sub-tables called **partitions**, all sharing the same schema. The parent table is a logical view; data is stored in the child partitions. Configured with `PARTITION BY RANGE`, `PARTITION BY LIST`, or `PARTITION BY HASH`.

Partition Key The column (or expression) whose value determines which partition a row belongs to. Queries that filter on the partition key allow the planner to skip irrelevant partitions via **partition pruning**.

Partition Pruning The planner's ability to exclude partitions that cannot contain rows matching a query's WHERE clause. Evaluated at plan time for constant predicates. Dramatically reduces I/O for queries that filter on the partition key. Pruning does not apply when the query contains no filter on the partition key.

Range Partitioning Partitions rows where the partition key falls within a defined range. Example: `FOR VALUES FROM ('2023-01-01') TO ('2024-01-01')`. Common for time-series data (dates, timestamps) where queries typically filter on a date range. Enables efficient archiving: drop the oldest partition instead of issuing a DELETE.

List Partitioning Partitions rows where the partition key matches a discrete set of values. Example: `FOR VALUES IN ('ES', 'FR', 'DE')`. Common for categorical data such as region, country, or status. A DEFAULT partition captures values not explicitly listed.

Hash Partitioning Distributes rows across N partitions by computing a hash of the partition key modulo N (`MODULUS N, REMAINDER R`). Produces an even distribution with no natural grouping. The planner can prune to a single partition for equality predicates on the partition key. Does not help range queries.

DEFAULT Partition A catch-all partition that receives any row whose partition key value matches no other defined partition. Prevents insert errors for unexpected values in list and range partitioned tables.

Sub-partitioning A partition that is itself partitioned. Enables composite strategies such as partitioning by year first and then by region within each year. Supported in PostgreSQL 10+.

DETACH PARTITION `ALTER TABLE parent DETACH PARTITION child` removes a partition from the partitioned table, converting it into a standalone regular table. The data is preserved. The detached table no longer appears in queries against the parent. Useful before dropping old data or archiving it.

ATTACH PARTITION `ALTER TABLE parent ATTACH PARTITION child FOR VALUES ...` adds an existing regular table as a new partition of a partitioned parent. PostgreSQL validates that all existing rows satisfy the partition constraint.

Partition Index Propagation In PostgreSQL 11+, an index created on a partitioned parent table is automatically created on every existing partition and on any future partition added with `ATTACH PARTITION` or `CREATE TABLE ... PARTITION OF`. In PostgreSQL 10, indexes had to be created manually on each partition.

Clustering

CLUSTER *(command)* Physically rewrites a table's heap so that rows are stored in the order defined by a specified index. After clustering, rows that share the same index key are located on adjacent heap pages, improving data locality for range scans. The index association is recorded so that `CLUSTER table_name` (without an index name) re-clusters using the same index.

Heap The main storage file for a table's rows. Rows are stored in pages (blocks, typically 8 kB) in the order they were inserted or last updated. Without clustering, rows for a given key value may be scattered across many different pages.

Data Locality The degree to which related rows are physically stored close together on disk. High data locality means a range query reads few pages; low locality means the same query must fetch many scattered pages. `CLUSTER` maximizes locality for one access pattern.

Cluster Decay The gradual degradation of the cluster order after new rows are inserted. New rows are appended to the heap in insertion order, not in the clustered order. Over time, rows for any given key value spread across new pages. Periodic re-clustering (or use of pg_repack) is needed to restore locality.

pg_repack A PostgreSQL extension that performs the same physical reordering as `CLUSTER` but without requiring an exclusive lock for the entire operation. It builds the reordered table in the background and swaps it in at the end with only a brief lock. Preferred over `CLUSTER` in production environments with live traffic.

Shared Buffer (shared hit / shared read) The PostgreSQL shared memory buffer pool. A `shared hit` in EXPLAIN BUFFERS means the page was already in the buffer pool (no disk I/O). A `shared read` means the page had to be read from disk. After clustering, the `shared read` count for a range query decreases because related rows share fewer pages.

Views and Materialization

View A named SQL query stored in the database catalog. Every time a view is queried, the underlying SQL is executed against current data. Views do not store data; they are transparent to the planner, which optimizes them together with the outer query (predicate pushdown applies).

Materialized View A view whose result is computed once and stored physically on disk. Queries against a materialized view read the stored snapshot instead of re-executing the underlying query. Requires explicit `REFRESH MATERIALIZED VIEW` to update the snapshot when underlying data changes.

REFRESH MATERIALIZED VIEW Re-executes the materialized view's defining query and replaces the stored snapshot. Acquires an exclusive lock on the view, blocking reads during the refresh.

REFRESH MATERIALIZED VIEW CONCURRENTLY Refreshes the materialized view by computing the new result and applying a diff to the stored snapshot, rather than replacing it entirely. Does not block concurrent reads. Requires a unique index on the materialized view.

General Database Concepts

MVCC (Multi-Version Concurrency Control) PostgreSQL's mechanism for allowing readers and writers to work concurrently without locking each other. Each transaction sees a consistent snapshot of the database at a point in time. Deleted or updated rows are not immediately removed; they become "dead tuples" visible to older transactions and cleaned up by VACUUM.

Dead Tuple A row version that is no longer visible to any active transaction (the result of a DELETE or UPDATE). Dead tuples occupy space until removed by VACUUM. A high ratio of dead tuples to live tuples is called **table bloat** and can slow down scans.

VACUUM A PostgreSQL maintenance process that removes dead tuples, reclaims space, and updates the visibility map (needed for Index Only Scans). Run automatically by the autovacuum daemon or manually with `VACUUM table_name`.

Autovacuum A background daemon that automatically runs VACUUM and ANALYZE on tables when their dead tuple counts cross configurable thresholds. Keeps tables healthy without manual intervention. Can be tuned per table with `ALTER TABLE ... SET (autovacuum_...)` storage parameters.

WAL (Write-Ahead Log) A sequential log of every change made to the database, written before the change is applied to the heap. Ensures durability (changes survive crashes) and enables replication. Operations that produce many WAL records (like large DELETEs) are heavier than metadata operations like dropping a table or partition.

DDL (Data Definition Language) SQL statements that define or modify database structure: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`. DDL operations like `DROP TABLE` are metadata operations - they update the catalog and remove the file reference, completing almost instantly regardless of table size.

DML (Data Manipulation Language) SQL statements that read or modify data: `SELECT`, `INSERT`, `UPDATE`, `DELETE`. DML operations are row-by-row (under MVCC) and scale with the number of affected rows.

Cardinality The number of distinct values in a column. High cardinality (e.g., a UUID primary key) means many unique values; low cardinality (e.g., a boolean or a `status` with three possible values) means few unique values. High-cardinality columns are better candidates for B-tree indexes because predicates on them are more selective.

Histogram A statistical summary stored in `pg_statistic` that divides the range of a column's values into buckets, recording the boundary value between each bucket. The planner uses histograms to estimate what fraction of rows satisfy a range predicate (e.g., `WHERE price BETWEEN 10 AND 50`).

random_page_cost A planner configuration parameter representing the estimated cost of fetching a non-sequential (random) page from disk. Default is `4.0`. For SSDs, setting it to `1.1` (close to `seq_page_cost = 1.0`) signals that random I/O is nearly as fast as sequential I/O, encouraging the planner to prefer index scans over sequential scans.

work_mem The amount of memory available to each sort or hash operation before spilling to disk. Higher `work_mem` allows larger hash tables and in-memory sorts. Setting it too low causes disk spills (`external sort` or `batches > 1` in EXPLAIN), which are much slower. Set per session for expensive queries: `SET work_mem = '256MB';`.

effective_cache_size A planner hint (not an actual allocation) indicating how much memory the OS and PostgreSQL buffer pool have available for caching data. A higher value makes the planner more confident that index pages will be found in cache, favoring index scans. Should be set to roughly half of total RAM.

Bulk Loading and Data Generation

CROSS JOIN Produces the Cartesian product of two tables: every row from the left table is paired with every row from the right table. If the left table has M rows and the right has N, the result has M × N rows. Used in data generation to enumerate all possible (user, game) or (user, achievement) combinations, then filtered to a random subset with `WHERE random() < p`. Because both source tables contain distinct rows, CROSS JOIN never produces duplicate pairs.

generate_series(start, stop) A set-returning function that produces one integer row for each value in the range `[start, stop]`. Used to generate N synthetic rows without needing a pre-existing source table. Example: `SELECT ... FROM generate_series(1, 50000) i` emits 50,000 rows numbered 1 through 50,000.

VOLATILE Function A function classified as `VOLATILE` may return a different result on every call, even with identical arguments. PostgreSQL cannot cache or eliminate calls to a VOLATILE function; it must evaluate it once per row. `random()` is VOLATILE, which guarantees that each row produced by a `CROSS JOIN` receives an independently sampled random value in the `WHERE` clause, making the probability filter correct.

random() A VOLATILE function that returns a uniformly distributed random number in the range [0, 1). Used as a probabilistic row filter: `WHERE random() < p` retains approximately a fraction `p` of all rows. Because `random()` is evaluated once per row, each combination in a CROSS JOIN is kept or discarded independently, producing a uniform random sample of all possible pairs.

synchronous_commit A PostgreSQL configuration parameter that controls when the server considers a transaction durable. The default (`on`) requires WAL to be flushed to disk before returning success to the client. Setting it to `off` skips the per-statement fsync: the WAL is still written but flushed to disk asynchronously in the background. If the server crashes shortly after a commit, the last few milliseconds of committed transactions may be lost — but the database itself remains consistent. For bulk data loading where durability of intermediate states is not required, `synchronous_commit = off` is the single highest-impact performance setting.

fsync The operating system call that flushes data from kernel write buffers to durable storage (disk). By default, PostgreSQL calls fsync at every `COMMIT` to guarantee that committed data survives a crash. Each

fsync adds latency. Wrapping many INSERT statements in a single explicit transaction reduces this to one fsync at the final `COMMIT`, and setting `synchronous_commit = off` defers even that flush.

Explicit Transaction (BEGIN / COMMIT) Wrapping multiple DML statements between `BEGIN` and `COMMIT` makes them a single atomic unit. For bulk loading this has two benefits: (1) one WAL fsync at `COMMIT` instead of one per statement (auto-commit mode), which can be several times faster; (2) if any statement fails, the implicit rollback leaves the table in a clean state rather than partially loaded.

*Custom GUC Parameter (SET app. / current_setting())** PostgreSQL allows user-defined configuration parameters under any custom namespace prefix, for example `app.num_users`. They are set with the standard SQL `SET app.num_users = '50000'` and read back with `current_setting('app.num_users')`. Unlike `\set` psql meta-commands (which expand as text before the query is sent to the server), custom GUCs are standard SQL and work identically in psql, pgAdmin, DBeaver, and application code. The value is always returned as `text`; cast to the required type with `::int`, `::numeric`, etc.

TRUNCATE ... RESTART IDENTITY CASCADE Extends `TRUNCATE` with two optional clauses:

- `RESTART IDENTITY` resets every SERIAL sequence associated with the truncated table back to its start value (usually 1), so the next insert begins numbering at 1 again.
- `CASCADE` automatically truncates any table that has a foreign key referencing the truncated table, preventing FK violation errors. Without `CASCADE`, PostgreSQL refuses to truncate a table that is referenced by another table's FK.

SERIAL / Sequence `SERIAL` is a PostgreSQL shorthand that creates an integer column backed by an auto-incrementing sequence object. Each `INSERT` that omits the column gets the next sequence value. Sequences increment even when a transaction is rolled back, which creates gaps between sequence values and the actual highest ID in the table. This is why using `MAX(id) + 1` to generate FK references is unsafe: the MAX may be lower than the sequence's next value, but it may also be lower than values already reserved by an in-flight transaction.